

EmvEnhanceRefactor - All Architecture Responses

Combined extract of prior architectural answers from the cloud-agent session.

Complete Architecture Analysis: Branch N vs Branch M

Branch N: cursor/transaction-step-dispatch-4c01

Branch M: cursor/package-layout-refactor-a435

Repository: essaylinkrefactor

Type: Senior architectural review (not a Git diff)

1. Branch N - Architecture

Narrative

N introduces PosTerminal as a reactive step dispatcher and EmvEngine as an abstract lifecycle (prepare / execute / complete). Host authorize and print live on PosTerminal.handleOnlineRequired / handleApproved.

But the composition root still builds the old stack (EmvTransaction + EmvBehavior + runners), while the UI already expects PosTerminal. So N is two architectures at once.

High-level flow

```
UI (MainActivity / MainViewModel)
  |
  | expects getTerminal()
  v
Intended: PosTerminal -> EmvEngine(prepare/execute/complete)
                  -> CommunicationBehavior / PrinterBehavior

Actually wired: EmvEnhanceApp -> EmvTransaction
                              -> EmvBehavior
                              -> Contact/Contactless Runners
                              -> Simple* Callback Adapters
```

Module relationship

```
:app (UI + vendors)
-> :core (ports + dual stacks)
-> :emvflow (runners/adapters)
-> PAX stack + WMRouter
```

Card search in N

Not a first-class port. Folded into EmvBehavior / runners / preprocess. PosTerminal does not search.

Strengths (N)

- Clear idea: sticky transaction stream + fire-and-forget EMV stream
- Host/print side-effects centralized on PosTerminal handlers
- Open/Closed intent via overridable handle* methods

Weaknesses (N)

- Dual unfinished stacks (largest issue)

- App/UI seam broken (getTransaction vs getTerminal)
- PaxEmvEngine stub; real PAX still on runners
- Fat unused handleEmv* surface
- Card search not vendor-portable as a contract
- README documents types that do not exist

2. Branch M - Architecture

Narrative

M completes a single coherent story:

- UI talks only to PosTerminal.
- Terminal owns card search + host + printer.
- EmvBehavior owns EMV after entry selection.
- EmvEngine is a thin event bus.

High-level flow

```
UI -> PosTerminal.acceptCard() / startTransaction()
    +-- EmvBehavior.prepare(engine, config)
    +-- PosTerminal.searchCard(config, listener)
    +-- EmvBehavior.start(engine, config, card)

EmvEngine - thin subjects + notify* -> EmvBehavior.dispatch*
```

Package organization

core (com.emvenhance.core)

Package	Contents
`terminal`	PosTerminal, EmvBehavior
`engine`	EmvEngine
`card`	EntryMethod, CardPresence, CardSearchListener, TransactionConfig
`event`	TransactionStep(Event), EmvStep(Event)
`host`	CommunicationBehavior, PrinterBehavior, AuthResult, HostDefaults

app vendors

```
TerminalFactory.create(VENDOR)
PAX      -> PaxTerminal -> PaxEmvBehavior
INGENICO -> IngenicoTerminal -> IngenicoEmvBehavior (stub)
FAKE     -> FakeTerminal -> FakeEmvBehavior
```

emvflow

runtime - preprocess - progress - device - pin

Card search in M

Owner: PosTerminal via searchCard + CardSearchListener.

Events: started - chip - contactless - mag - manual - removed - timeout - cancelled - error.

Object ownership (M)

```

PostTerminal owns:
+-- EmvEngine
+-- EmvBehavior
+-- CommunicationBehavior -> authorize()
+-- PrinterBehavior       -> printReceipt()

```

Strengths (M)

- One public API; UI/core never import vendors
- Card search is a real, portable contract
- Engine is genuinely thin
- Host/printer owned by terminal (correct layer)
- Package boundaries match responsibilities
- Adding a vendor is mechanical
- Dual stacks / runners / callback adapters removed from app path

Weaknesses (M)

- PAX runtime/WMRouter still under the hood
- FAKE/INGENICO Gradle still pull PAX modules
- PaxEmvBehavior is a large multi-interface class
- Ingenico is structural stub, not a second real SDK proof
- Async DAL race on first PAX transaction
- Host still passed into behaviors for online
- Unused CardSearchListener.Adapter

3. Side-by-side Comparison

Category	N	M	Winner	Why
Overall architecture	4/10	9/10	M	N is two incomplete stacks; M is one coherent flow
Separation of concerns	5/10	9/10	M	Search/host/printer/EMV/engine cleanly split in M
Module boundaries	6/10	8/10	M	Core packages in M; both still Gradle-coupled to PAX
Package organization	4/10	9/10	M	N flat + dual models; M intentional packages
Vendor independence	4/10	8/10	M	N folds search into engine/runners; M has searchCard + factory
Extensibility	5/10	9/10	M	Add vendor without touching engine/UI in M
Scalability	5/10	8/10	M	M scales by vendor plug-ins
SOLID	4/10	8/10	M	N violates ISP + unfinished DIP composition
Event architecture	7/10	8/10	M	Same dual-stream idea; M completes wiring

Transaction lifecycle	5/10	9/10	M	Clear prepare->search->start
Card search design	3/10	9/10	M	No a port in N; first-class in M
Host/printer placement	7/10	9/10	M	M finishes ownership without AbstractEmvBehavior
Simplicity	3/10	8/10	M	N has EmvTransaction + EmvEngine + runners + adapters
Readability	4/10	8/10	M	M's mental model fits one diagram
Testability	5/10	8/10	M	FakeTerminal isolates without runners
Performance design	6/10	7/10	M	Similar IO-thread blocking; M removes adapter hops
Maintainability	3/10	8/10	M	N mid-migration is high risk
Composition root	2/10	9/10	M	N App/UI mismatch is an architectural break
Weighted overall	**~4.5**	**~8.3**	**M**	

Where N is better: The dual event-stream concept and "terminal owns side effects" originated clearly in N.

Where M is superior: Nearly everything else - composition completeness, card-search abstraction, removal of dual stacks.

4. Best-of-Both Recommendation

Do not ship N. Take M as the base, keep N's best ideas, cut remaining debt.

Rules

1. One stack only - prepare -> search -> start
2. Keep dual event streams (sticky transaction + publish EMV)
3. Card search on PosTerminal
4. Host/printer on PosTerminal
5. No AbstractEmvBehavior auto online/print
6. No empty handler forest
7. Vendor = Terminal + EmvBehavior + factory line
8. Gradle flavors - PAX modules only on PAX flavor
9. Online via terminal.authorize(config) long-term
10. Print as UI/terminal policy, not inside EmvBehavior
11. Delete unused Adapter unless needed
12. Prove second vendor with real SDK

Final verdict

Question	Answer
Superior overall?	**Branch M** , clearly
Is N worthless?	No - dual events + terminal side-effects
Merge N as-is?	**No**
Production base?	**M** + flavor isolation + real second vendor
Biggest M risk?	PAX SDK/runtime coupling, unproven Ingenico

Bottom line: Branch N sketched the reactive shell but left two competing designs. Branch M finished the architecture around PosTerminal as the single vendor-agnostic API. Ideal = M's structure + N's event insight ? leftover PAX compile coupling ? unused ceremony.

PosTerminal-Centric Architecture Refactor

Branch: [cursor/full-postterminal-refactor-a435](#) / continued in package-layout
PR: <https://github.com/berothomas33/esaylinkrefactor/pull/3>

Goal

Make one PosTerminal API work with any card on any vendor.
UI and core never import Pax / Ingenico / Fake types.

End-state architecture

```
UI -> PosTerminal.acceptCard() / startTransaction()
    +-- EmvBehavior.prepare()
    +-- searchCard(config, CardSearchListener) <- vendor readers
    +-- EmvBehavior.start(engine, config, card) <- vendor EMV

EmvEngine - thin subjects + notify* -> behavior.dispatch*
```

Layer	Owns
`PosTerminal`	Device init, unified card search, cancel, host, printer
`EmvBehavior`	EMV after entry method selected
`EmvEngine`	Reactive events for UI (no business logic)

What changed

Unified card search

- Added EntryMethod, CardSearchListener, CardPresence
- Removed CardReader
- PosTerminal orchestrates: prepare -> searchCard(listener) -> behavior.start(card)
- Vendor terminals fire listener events from native SDKs

Card search listener events

- Search started
- Chip (ICC) detected
- Contactless (NFC/PICC) detected
- Magstripe detected
- Manual entry selected
- Card removed
- Search timeout / cancelled

- Reader error

Vendor layout

```
vendor/  
  TerminalFactory.java  
  pax/      PaxTerminal, PaxEmvBehavior, PaxKernel  
  fake/     FakeTerminal, FakeEmvBehavior  
  ingenico/ IngenicoTerminal, IngenicoEmvBehavior (stub)
```

UI

- Primary action: Present card (any) -> PosTerminal.acceptCard() (Mode.ANY)
- Optional chip-only / contactless-only
- Cancel

Removed layers

- Per-vendor host/printer stub classes (later moved to PosTerminal ownership)
- Router service lookup from app vendor façade
- Contactless/Contact runners from app path
- Callback adapters

Adding a vendor

1. Implement XxxTerminal extends PosTerminal (searchCard, cancelCardSearch)
2. Implement XxxEmvBehavior implements EmvBehavior
3. One line in TerminalFactory
4. No core / engine / UI changes

Ingenico note

No Ingenico SDK was attached in the workspace. Stubs demonstrate the extension pattern; replace with Tetra/Axium when available. Wire with -PVENDOR=INGENICO.

Verification

:app:compileDebugJavaWithJavac succeeded for FAKE, INGENICO, and PAX.

Package Layout Refactor

Branch: [cursor/package-layout-refactor-a435](#)
PR: <https://github.com/berothomas33/esaylinkrefactor/pull/4>

Goal

Arrange packages by responsibility across core, emvflow, emvservice, and emvbase. Behavior unchanged - structure and import cleanup.

`:core` -> `com.emvenhance.core.\*`

Package	Contents
`terminal`	`PostTerminal`, `EmvBehavior`
`engine`	`EmvEngine`
`card`	`EntryMethod`, `CardPresence`, `CardSearchListener`, `TransactionConfig`
`event`	`TransactionStep(Event)`, `EmvStep(Event)`
`host`	`CommunicationBehavior`, `PrinterBehavior`, `AuthResult`, `HostDefaults`

`:emvflow` -> `com.emvenhance.emvflow.\*`

Package	Contents
`runtime`	`EmvFlowRuntime` (DAL / WMRouter lazy init)
`preprocess`	`EmvPreProcessFacade`
`progress`	`EmvStepProgress`
`device`	`EmvDeviceImpl`, cipher mode
`pin`	`IPinTask`

`:emvservice:export` -> `com.pax.emvservice.export.\*`

Package	Contents
`api`	`IEmvBase`, `IEmvCallback`, `IEmvCardInfoService`, ...
`contact` / `contactless`	Contact / CLSS service + result listeners
`mag` / `manual`	Magstripe / manual entry APIs
`pin` / `version` / `constant` / `exceptions`	Supporting APIs

Router keys unchanged (still string keys + interface class).

`:emvservice:emv` -> `com.pax.emvservice.emv.\*`

Package	Contents
`init`	`EmvInit`
`contact` / `contactless` / `mag` / `manual` / `pin` / `version`	Concrete Router services

`:emvbase` -> `com.pax.emvbase.\*`

Already structured; documented with package-info:

- constant - tags / constants
- param - AID / CAPK / terminal params (common / contact / clss)

- process - contact / contactless callbacks and results
 - utils - helpers
-

`:app` vendors

```
vendor.TerminalFactory  
vendor.pax / vendor.fake / vendor.ingenico
```

Verification

:core, :emvflow, :emvservice:*, :emvlib, :app compile OK.

EmvBehavior vs Host/Printer Ownership

Branch: cursor/package-layout-refactor-a435

Related commits: Remove AbstractEmvBehavior; Own Communication/Printer on PosTerminal

Problem

AbstractEmvBehavior held CommunicationBehavior and PrinterBehavior and auto-ran:

- onOnlineRequired -> host authorize
- onApproved -> print receipt

That mixed EMV lifecycle with host/printer side services. The class name EmvBehavior did not match those jobs.

Decision

Removed

- AbstractEmvBehavior entirely
- Auto host authorize / auto print from EMV layer
- Temporary removal of unused host stubs (later restored on PosTerminal)

Correct ownership

```
PosTerminal  
+-- CommunicationBehavior -> authorize()  
+-- PrinterBehavior        -> printReceipt()  
+-- searchCard(...)        ->  
+-- EmvBehavior            -> EMV only
```

Rules

1. EmvBehavior = vendor EMV after entry selection (prepare / start / cancel)
2. PosTerminal = owns readers, host, printer
3. Vendors may use the same CommunicationBehavior instance for online

4. Printer stays on the terminal - not inside EmvBehavior

Why this is better

Concern	Wrong place	Right place
Card search	Engine / EmvBehavior	PosTerminal
Host authorize	AbstractEmvBehavior	PosTerminal
Receipt print	AbstractEmvBehavior.onApproved	PosTerminal.printReceipt
EMV kernel flow	-	EmvBehavior

Current wiring

```
protected PosTerminal(  
    EmvEngine engine,  
    EmvBehavior behavior,  
    CommunicationBehavior communication,  
    PrinterBehavior printer)
```

Public helpers:

- authorize(TransactionConfig)
- printReceipt(List<String>)
- communication() / printer()

Vendor example:

```
PaxTerminal  
-> HostDefaults.declineUntilWired() + logPrinter  
-> PaxEmvBehavior(kernel, communication) // same communication instance
```

Design Q&A and Layer Summary

Extracted from conversation answers on the EmvEnhance refactor.

Q1 - What enhanced the layer model?

PosTerminal is the only public API - it owns card search for every vendor.

EmvBehavior runs EMV only after an entry method is chosen (chip / tap / swipe / manual).

EmvEngine is just an event bus - no business logic.

Vendors live in separate packages behind TerminalFactory; unused host stubs, adapters, and reporters were removed from the wrong layers.

Q2 - Why were CommunicationBehavior and PrinterBehavior inside AbstractEmvBehavior?

They were there for convenience: shared defaults for online authorize and print after EMV steps.

They did not match the job description of EmvBehavior (vendor EMV lifecycle). Host and printer are side services. Correct place: PosTerminal.

Q3 - Do we need AbstractEmvBehavior online/print methods?

No. Those methods were removed:

- onOnlineRequired auto-authorize
- onApproved -> print
- deliverOnlineResult / buildReceipt abstract helpers

AbstractEmvBehavior was deleted. Vendors implement EmvBehavior directly.

Q4 - Where do CommunicationBehavior and PrinterBehavior go?

In abstract PosTerminal:

```
PosTerminal
+-- CommunicationBehavior
+-- PrinterBehavior
+-- searchCard(...)
+-- EmvBehavior (EMV only)
```

Q5 - What is CardSearchListener.Adapter?

A convenience no-op base for CardSearchListener.

The interface has many methods. Adapter implements all as empty so a caller can override only what they care about.

In this project it is unused - PosTerminal uses a full EngineReportingListener. Safe to delete if desired.

Final layer map

Layer	Role
UI	Display; calls `acceptCard` / cancel
PosTerminal	Search, host, printer, orchestration
EmvBehavior	Vendor EMV after entry selected
EmvEngine	Subjects + notify/dispatch
TerminalFactory	Vendor selection only
emvflow	PAX runtime helpers (not core)

Branches / PRs produced

Branch	Focus	PR
`cursor/pax-emv-behavior-bridge-a435`	PaxEmvBehavior bridge, searchCard listener	#2

`cursor/full-postterminal-refactor-a435`	Full PosTerminal-centric refactor	#3
`cursor/package-layout-refactor-a435`	Packages + host on PosTerminal	#4
`cursor/transaction-step-dispatch-4c01`	Earlier step-dispatcher (Branch N baseline)	base